

Visual Computing Project Report

Tim Dedie & Nick Somitsch

January 20, 2026

Introduction

Task. We built a system that recognizes hand gestures from a webcam feed. It classifies five gestures: thumbs up, thumbs down, peace sign, open palm, and no hand (Figure 1).

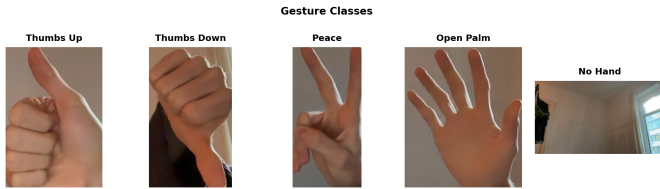


Figure 1: The five gesture classes our system recognizes.

Problem. Our first attempts didn't work well. We fed full webcam frames into the network and the model just learned to recognize the background instead of the actual hand gestures. If we recorded thumbs up in front of a bookshelf, the model associated "bookshelf" with thumbs up. This was frustrating until we realized we needed to crop out just the hand first.

Solution. We use Google's MediaPipe to detect and crop the hand region, then feed that cropped image to a CNN for classification. With this approach and the right augmentation settings, we got 100% accuracy on our test set.

Contributions. Tim: data collection and augmentation experiments. Nick: CNN implementation, training scripts, and live demo.

Background

Why MediaPipe? MediaPipe Hands is a pre-trained model from Google that detects 21 landmarks on a hand (fingertips, knuckles, palm, etc.). We use these points to draw a bounding box around the hand and crop it out. This way our CNN only sees the hand, not the background.

Our CNN architecture. We use a simple CNN with three convolutional blocks:

- Input: 64×64 RGB image (3 channels)
- Conv1: $3 \rightarrow 32$ filters, 3×3 kernel, ReLU, Max-Pool $\rightarrow$ output 32×32
- Conv2: $32 \rightarrow 64$ filters, 3×3 kernel, ReLU, Max-Pool $\rightarrow$ output 16×16
- Conv3: $64 \rightarrow 128$ filters, 3×3 kernel, ReLU, MaxPool $\rightarrow$ output 8×8
- Flatten: $128 \times 8 \times 8 = 8192$ features
- FC1: $8192 \rightarrow 256$, ReLU, Dropout (50%)
- FC2: $256 \rightarrow 5$ (one per gesture class)

Total: about 2.19 million parameters. Figure 2 shows the full architecture.

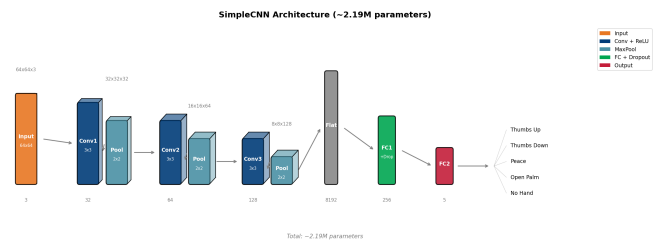


Figure 2: Our SimpleCNN architecture.

Loss function. We use softmax and cross-entropy:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}, \quad \mathcal{L} = - \sum_i y_i \log p_i \quad (1)$$

Augmentations. We implemented these ourselves with OpenCV (no torchvision):

- Horizontal flip – mirrors the image
- Rotation – random angle between -30 and $+30$

- Brightness – multiply by random factor 0.5 to 1.5
- Gaussian blur – kernel size 3, 5, or 7
- Color jitter – shift hue and saturation in HSV
- Zoom – scale up and crop center

Each augmentation is applied with 50% probability during training. Figure 3 shows examples of each augmentation applied to the same image.

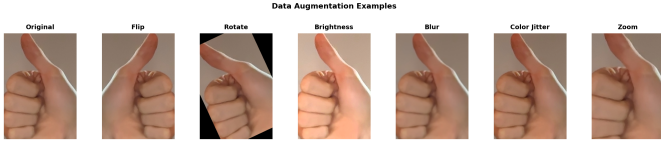


Figure 3: Examples of our data augmentations.

Method

Figure 4 shows our complete pipeline from webcam input to gesture classification.

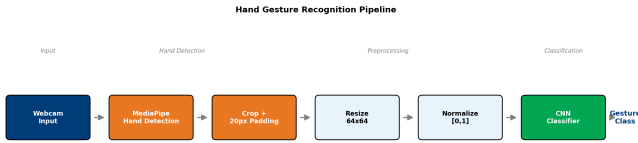


Figure 4: Our hand gesture recognition pipeline.

Data collection. We wrote a script that:

1. Captures webcam frames
2. Runs MediaPipe to find the hand
3. Crops the hand region with 20px padding
4. Saves to folders (one per gesture class)

We collected around 300 images per class. For "no hand" we just saved the full frame.

Preprocessing pipeline:

1. Load image with OpenCV
2. Apply augmentation (50% chance, training only)
3. Resize to 64×64
4. Normalize to $[0, 1]$
5. Convert from HWC to CHW format

Training setup:

- Optimizer: Adam, learning rate 0.001
- Loss: CrossEntropyLoss
- Epochs: 20
- Batch size: 32
- Train/test split: 80/20, stratified, seed 42

Figure 5 shows how the models learn over time. All configurations converge within 20 epochs.

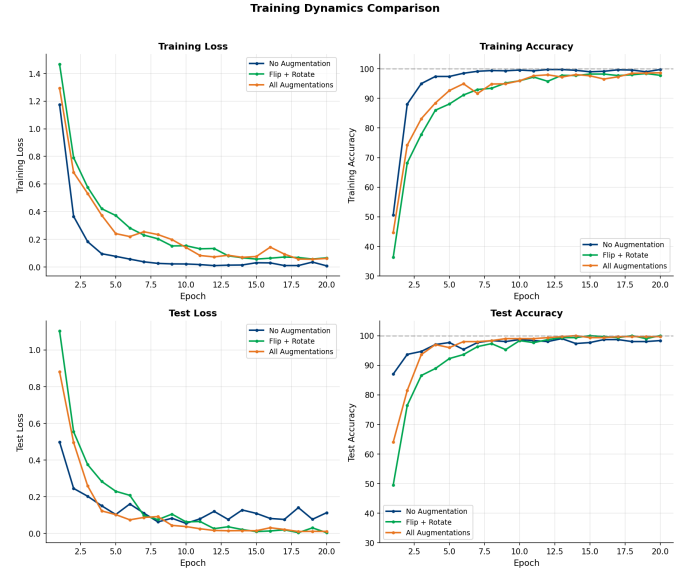


Figure 5: Training and test loss/accuracy over 20 epochs.

Experiments. We ran two sets of experiments:

1. *Augmentation comparison:* no augmentation, single augmentations (flip, rotate, brightness, blur), flip+rotate combined, all augmentations
2. *Dataset size:* 50, 100, 200, and full samples per class, with and without augmentation

Results

Augmentation results. Table 1 and Figure 6 show that flip+rotate gives the best results. Even better than using all augmentations together.

Table 1: Test accuracy by augmentation

Config	Accuracy
None	98.33%
Flip only	96.33%
Rotate only	97.67%
Brightness only	98.00%
Blur only	98.67%
Flip + Rotate	100.00%
All	99.67%

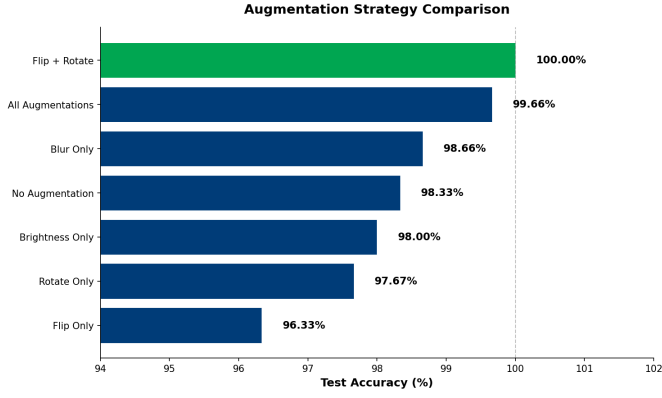


Figure 6: Test accuracy comparison across augmentation strategies. Flip+rotate achieves 100% accuracy.

Figure 7 shows the confusion matrix for our best model (flip+rotate). No misclassifications.

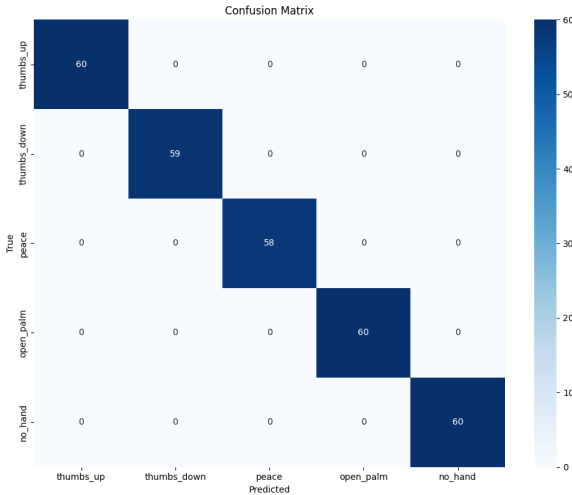


Figure 7: Confusion matrix for flip+rotate model.

Dataset size results. Table 2 and Figure 8 show that augmentation helps a lot when you have little data (90% $\rightarrow$ 98% with only 50 samples). With enough data, augmentation matters less.

Table 2: Test accuracy by dataset size

Samples/class	No Aug	With Aug
50	90.00%	98.00%
100	98.00%	97.00%
200	99.00%	97.50%
Full (~300)	100.00%	99.67%

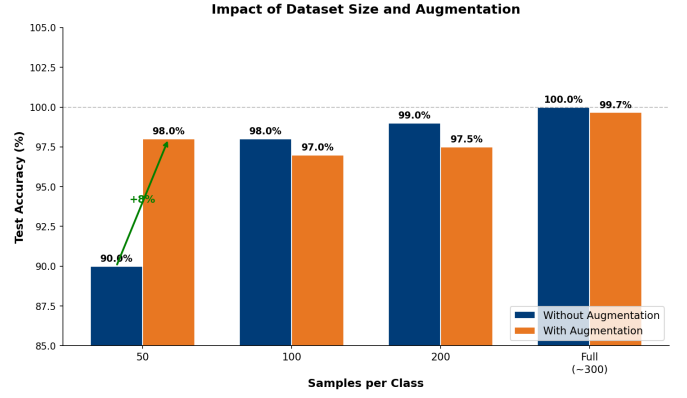


Figure 8: Impact of dataset size on accuracy. Augmentation provides an 8% boost with only 50 samples per class.

Conclusion

What worked. The biggest improvement came from adding hand cropping with MediaPipe. Before that, the model was basically useless. The flip+rotate augmentation combo worked best, probably because gestures can appear at different angles and orientations.

What we struggled with.

- Initially had no idea why accuracy was bad. Turned out the model was learning backgrounds
- MediaPipe version issues (newer versions removed the API we needed, had to pin to 0.10.9)
- More augmentations didn't mean better results. All augmentations actually performed slightly worse than just flip+rotate

Limitations.

- Only 5 gestures
- Single hand only
- Needs decent lighting for MediaPipe to work
- Trained on our hands only. Might not generalize to other people